

Memorial Técnico de Desenvolvimento

Portal de Solicitações Internas — Seleção DEV Jr. 09/2026 (bit Soluções)

1. Tecnologias Utilizadas

- **Linguagens:** PHP 8.4, TypeScript, HTML, CSS, SQL e YAML (arquivos do Docker Compose).
- **Framework backend:** Laravel 13.
- **Framework frontend:** Vue 3, com Vite 8 (build e servidor de desenvolvimento), Vue Router 5 (navegação), Pinia 4 (estado), Axios (cliente HTTP) e Tailwind CSS 4 (estilo).
- **Banco de dados:** PostgreSQL 17, com a estrutura criada por migrations do Laravel.
- **Autenticação:** Laravel Sanctum 4, no modo SPA (sessão em cookie com proteção CSRF).
- **Validação:** Form Requests do Laravel no backend, com regra própria de CPF; validação dos formulários também na SPA.
- **Containerização:** Docker e Docker Compose (um arquivo para desenvolvimento e outro para produção), com nginx como entrada única da aplicação.
- **Testes:** PHPUnit na API e Vitest com Vue Test Utils na SPA.
- **Qualidade de código:** Laravel Pint (padrão do PHP), ESLint, Oxlint, Prettier e vue-tsc (checagem de tipos).
- **Serviços cloud:** nenhum. A aplicação roda inteira em containers, localmente.

2. Justificativa Técnica

Laravel (backend)

Escolhi o Laravel para a API pela agilidade e simplicidade que o projeto pedia, num prazo de cerca de cinco dias. O framework já traz pronto o que o desafio exige: rotas, migrations e seeders para criar o banco, Eloquent para o acesso aos dados, Form Requests para validação, Policies para autorização e tratamento de erros em JSON. Com isso, o tempo foi para as regras de negócio e não para montar infraestrutura.

Em relação a alternativas como Spring Boot ou Django, o Laravel exige menos configuração para uma API desse porte e tem convenções fortes, o que deixa o código previsível para quem for manter. Se o sistema precisasse de mais desempenho e robustez, com alto volume de requisições ou processamento pesado, eu consideraria outra ferramenta, como o Spring Boot. Para um portal interno, o Laravel atende bem e escala horizontalmente, já que o php-fpm não guarda estado entre requisições e a sessão fica no banco.

Vue.js 3 com TypeScript (frontend)

Escolhi o Vue.js para a SPA porque ele já vem com as ferramentas essenciais oficiais: o Vue Router para as páginas e o Pinia para o gerenciamento de estado. No React, por exemplo, essas peças precisam ser escolhidas entre várias bibliotecas da comunidade. A Composition API permite separar a lógica de tela em composables reutilizáveis, e o TypeScript tipa os contratos com a API, o que pega erros de integração antes de rodar.

Vite, Tailwind CSS e Axios

O Vite é o build padrão do Vue e recarrega a tela na hora durante o desenvolvimento, o que acelera a produtividade. O Tailwind CSS evita arquivos de CSS paralelos e facilitou o modo escuro e a responsividade. O Axios foi escolhido pelos interceptadores: um único ponto trata sessão expirada (401), token CSRF vencido (419, com nova tentativa automática) e troca de senha obrigatória (403), sem repetir esse código em cada tela.

PostgreSQL

O PostgreSQL oferece transações confiáveis e bloqueio de linha (SELECT ... FOR UPDATE), que usei para evitar que duas pessoas mudem a situação da mesma solicitação ao mesmo tempo. Também tem o operador ILIKE, que faz a busca por título sem diferenciar maiúsculas e minúsculas. Em relação ao MySQL, a integração com o Laravel é equivalente, e o PostgreSQL é mais rigoroso com tipos e integridade.

Laravel Sanctum

O Sanctum no modo SPA autentica por sessão em cookie httpOnly, com proteção CSRF. Isso é mais seguro do que guardar um token JWT no localStorage, onde um script malicioso poderia lê-lo. Em relação ao Passport ou a JWT, o Sanctum é bem mais simples e suficiente para uma SPA que fala com a própria API.

Docker, Docker Compose e nginx

Com Docker, quem avalia não precisa instalar PHP, Node ou PostgreSQL: um *docker compose up* sobe tudo, roda as migrations e os seeders. O nginx fica na frente da API e da SPA numa mesma origem (localhost:8080), o que elimina a configuração de CORS e os problemas de cookie entre portas diferentes. Há um compose de desenvolvimento, com o código montado e recarga automática, e um de produção, com imagens fechadas e containers somente leitura.

PHPUnit, Vitest e Vue Test Utils

São as ferramentas padrão de cada ecossistema: o PHPUnit já vem com o Laravel, e o Vitest usa a mesma configuração do Vite. Os testes da API rodam num banco PostgreSQL separado, descartável, para nunca apagar os dados de desenvolvimento. Os testes dão segurança para refatorar, o que impacta diretamente a manutenção.

3. Justificativa Conceitual

3.1 Estrutura geral da aplicação

O projeto é um monorepo com três pastas principais: **api/** (backend Laravel), **web/** (frontend Vue) e **docker/** (Dockerfiles, entypoints e configuração do nginx). O navegador acessa tudo pelo nginx: os caminhos `/api` e `/sanctum` vão para o PHP, e o restante vai para a SPA.

3.2 Organização das camadas

No backend, cada requisição passa por camadas com uma responsabilidade cada:

- **Rotas e middlewares:** exigem sessão, bloqueiam usuário desativado e bloqueiam o sistema enquanto a senha inicial não for trocada.
- **Form Requests:** validam a entrada e devolvem 422 com a mensagem de cada campo.
- **Policies:** decidem quem pode o quê, aplicadas direto nas rotas.
- **Controllers:** são finos, só recebem a requisição e chamam o serviço.
- **Services:** concentram as regras de negócio (DemandService, AuthService, UserService, DashboardService).
- **Models e Enums:** representam as tabelas e os valores fixos (situação, categoria, perfil e tipo de movimentação).
- **Resources:** definem o JSON de saída, expondo só o necessário (a listagem mostra o nome do solicitante, mas não o CPF nem o e-mail).

No frontend, a divisão é: **views** (telas), **components** (peças reutilizáveis, separadas por domínio), **services** (um arquivo de chamadas à API por domínio, todos sobre o mesmo cliente HTTP), **stores** (estado global com Pinia), **composables** (lógica de tela reutilizável) e **router** (rotas e guarda de autenticação).

3.3 Estratégia de modelagem de dados

O banco tem três tabelas de negócio:

- **users:** nome, usuário, CPF, e-mail, senha, perfil (admin ou colaborador), se está ativo e se precisa trocar a senha.
- **demands:** título, descrição, categoria, situação, solicitante (chave estrangeira para users), datas de criação e alteração, e exclusão lógica (soft delete).
- **demand_movements:** o histórico de cada solicitação, com o tipo do ato (criada, editada, iniciada, finalizada, reaberta), quem fez e quando. É uma tabela só de inserção.

Categoria e situação são colunas de texto validadas por enums do PHP, e não enums do PostgreSQL, para que

uma categoria nova não exija alterar o tipo no banco. Usuários nunca são apagados, só desativados; as chaves estrangeiras usam RESTRICT como proteção extra. A situação segue o fluxo Pendente (Aberto) → Em andamento (Em Atendimento) → Finalizada (Concluído), com a opção de reabrir. Editar e excluir só são permitidos enquanto a solicitação está pendente.

3.4 Padrões de projeto utilizados

- **Service Layer:** regras de negócio fora dos controllers.
- **Policy:** autorização centralizada, testável e separada da regra de negócio.
- **Form Request e Resource:** validação na entrada e formatação na saída, nas bordas da API.
- **Máquina de estados simples:** cada ação (iniciar, finalizar, reabrir) só parte de uma situação e só leva a outra; fora disso, a API responde 422.
- **Transação com bloqueio pessimista:** a mudança de situação e o registro no histórico acontecem juntos; se um falha, o outro é desfeito.
- **Store e Composable** no frontend, e **Interceptor** no cliente HTTP.

3.5 Estratégia de autenticação

O login aceita usuário ou e-mail e senha, com a opção "Lembrar-me". A sessão é mantida em cookie pelo Sanctum. Algumas medidas de segurança básica:

- Limite de tentativas: 5 erros por minuto para o mesmo login e IP, e 30 por IP.
- Mesma mensagem e mesmo tempo de resposta para usuário inexistente e senha errada, para não revelar quem tem conta.
- Troca de senha obrigatória no primeiro acesso, imposta pela API, e não só pela tela.
- Usuário desativado perde o acesso na requisição seguinte, mesmo com sessão aberta.
- Dois perfis: o administrador cadastra e gerencia usuários; o colaborador registra e atende solicitações.

3.6 Comunicação entre frontend e backend

A comunicação é por API REST em JSON, sob o prefixo /api. Os principais endpoints são: login, logout e /me; CRUD de /demands, com /demands/search para filtros e as ações /start, /close e /reopen; /dashboard/summary, /categories e /trend; e /users para a administração. As listagens são paginadas. Os erros seguem um formato único em JSON (401, 403, 404, 419, 422 e 429), e a SPA trata cada um no interceptor ou mostra a mensagem do campo no formulário.

3.7 Organização do código-fonte

No código, os nomes são em inglês e seguem o padrão de cada framework. A entidade se chama *Demand*, e não *Request*, para não confundir com a classe Request do Laravel; na tela, o nome é "Solicitação". O histórico do Git usa mensagens no padrão feat/fix/refactor, e o trabalho foi feito em branches de funcionalidade integradas por pull request. As principais decisões arquiteturais (entrada única pelo nginx, .env único na raiz e quem pode atender solicitações) foram registradas antes da implementação, junto com a especificação de cada funcionalidade.

4. Análise Crítica

4.1 O que mudei em relação aos requisitos

Alguns requisitos eu mudei por vontade própria. A tabela de usuários tem nome, CPF, e-mail, perfil, se está ativo e se precisa trocar a senha, além do usuário e da senha propostos. Nas solicitações, coloquei a chave estrangeira requester_id, que aponta para o usuário solicitante. Criei também a tabela demand_movements, que liga cada solicitação aos usuários que agiram nela e guarda o histórico completo. Acrescentei ainda uma tela de administração de usuários e, no dashboard, a divisão por categoria e a evolução no tempo, além dos quatro indicadores pedidos.

4.2 Limitações da solução

Por ser um projeto feito em pouco tempo, existem limitações que afetam a escalabilidade e o uso real:

- Não há setores entre os usuários. Qualquer colaborador pode atender qualquer solicitação, sem separação de quem pode manipular cada demanda.
- As categorias são fixas no código (enum), e não uma tabela. Para criar uma categoria nova, é preciso mexer no código e publicar de novo.
- Não há comentários, prazo, prioridade nem SLA para dar mais informações sobre o andamento da solicitação.
- O histórico registra o tipo do ato e quem fez, mas não quais campos foram alterados numa edição.
- Não há anexos nem notificações (e-mail ou aviso na tela) quando a situação muda.
- Todo usuário novo recebe a mesma senha inicial. O risco é reduzido pela troca obrigatória e pelo limite de tentativas por IP, mas continua existindo.
- Não há CI/CD, e o ambiente de produção roda só localmente, sem HTTPS.
- A busca por texto é feita só no título, com ILIKE, o que não escala bem para grandes volumes.

4.3 Melhorias futuras

- Criar uma tabela de setores, ligar usuários e categorias a eles e limitar o atendimento ao setor responsável.
- Transformar as categorias em tabela gerenciada pelo administrador.
- Adicionar comentários, prioridade, prazo e indicadores de SLA no dashboard.
- Registrar no histórico o antes e o depois de cada edição.
- Enviar notificações por e-mail usando as filas do Laravel.
- Criar um pipeline de CI (por exemplo, GitHub Actions) que rode testes, lint e checagem de tipos a cada push.
- Usar busca textual completa do PostgreSQL (full-text search) no título e na descrição.

4.4 Requisitos que poderiam ser aperfeiçoados

- O documento não diz quem pode alterar o status. Decidi que qualquer colaborador pode atender, e só quem pediu (ou o administrador) pode editar e excluir.
- Não há previsão para reabrir uma solicitação concluída; incluí essa ação por ser comum na prática.
- "Excluir" não define se a exclusão é física ou lógica. Usei exclusão lógica, para preservar o histórico.
- O cadastro de usuários não é citado, mas sem ele o sistema só teria o administrador.

4.5 O que seria diferente num ambiente corporativo de produção

- Login integrado ao diretório da empresa (LDAP, Active Directory ou SSO), em vez de senhas locais e senha inicial padrão.
- Setores e permissões por setor, como descrito acima.
- Deploy em nuvem ou servidor próprio com HTTPS, backup automático do banco e banco gerenciado.
- Monitoramento e logs centralizados, além do endpoint de health que já existe.
- CI/CD com publicação automática depois que os testes passam.
- Se o volume de uso exigisse mais desempenho e robustez, eu avaliaria outra stack no backend, como o Spring Boot.